Universidad del Valle de Guatemala

Facultad de Ingeniería

Catedrático: Kevin Antonio Velásquez Aguilar

Programación Orientada a Objetos

Ejercicio 5 – Polimorfismo vía herencia

Nombre: María Isabel Cajas Cuéllar

Carné: 26183

Sección: 20

Fecha de entrega: 09/10/2026

1. Requisitos funcionales
    - Cargar los módulos iniciales: “Mision” crea los diez módulos con sus datos iniciales.
    - Guardar los módulos: “Mision” los almacena en una lista polimórfica.
    - Listar los módulos: “Mision” muestra todos los módulos y sus características.
    - Buscar un módulo: “Mision” lo localiza por su ID o nombre.
    - Ordenar los módulos: “Mision” genera un catálogo ordenado por costo.
    - Avanzar un ciclo: “Mision” permite que todos los módulos intenten operar en orden.
    - Administrar los recursos: “EstadoMision” controla la energía y los datos.
    - Generar energía: “ModuloEnergia” aumenta la energía disponible.
    - Recolectar datos: “ModuloVuelo” consume energía y agrega datos en órbita.
    - Descargar datos: “ModuloTierra” consume energía y descarga los datos disponibles.
    - Informar operaciones fallidas: los módulos indican si no operaron y explican la causa.
    - Mantener los resultados: “Mision” conserva la energía y los datos para el siguiente ciclo.
    - Generar el resumen de comunicaciones: “Mision” calcula la información de los módulos de tierra.
    - Mostrar el menú: “VistaConsola” presenta las opciones y los resultados.
    - Controlar las opciones: “ControladorMision” ejecuta la operación seleccionada.
    - Iniciar el programa: App crea el modelo, la vista y el controlador.

2. Validaciones y Restricciones

    - Opción numérica: “VistaConsola” evita que el programa falle si el usuario escribe texto en el menú.
    - Opción válida del menú: “ControladorMision” informa cuando se ingresa una opción diferente de 0 a 5.
    - Módulo activo: “ModuloEnergia”, “ModuloVuelo” y “ModuloTierra” comprueban que el módulo esté activo antes de operar.
    - Energía suficiente para recolectar: “ModuloVuelo” no opera si no dispone de toda la energía requerida.
    - Sin recolecciones parciales: “ModuloVuelo” recolecta la cantidad completa de datos o no recolecta nada.

- Energía suficiente para descargar: “ModuloTierra” no descarga si no puede cubrir todo su consumo energético.
- Datos pendientes: “ModuloTierra” solamente descarga cuando existen datos en órbita.
- Descarga limitada: “ModuloTierra” no descarga más datos que los disponibles ni más que su capacidad.
- Consumo completo por descarga: “ModuloTierra” consume toda la energía requerida, aunque descargue menos de su capacidad máxima.
- Módulo inexistente: “ControladorMision” informa cuando no se encuentra un módulo con el ID o nombre indicado.
- Capacidades constantes: “ModuloEnergia”, “ModuloVuelo” y “ModuloTierra” utilizan atributos “final para sus capacidades.
- Acumulación válida: “ModuloEnergia” y “ModuloVuelo” incrementan sus ciclos acumulados únicamente cuando realizan su operación.
- Conservación del estado: “Mision” mantiene el mismo objeto “EstadoMision” para que los recursos no se reinicien.
- Orden original: “Mision” ordena una copia de la lista para no modificar el orden de participación de los módulos.

3. Clases y responsabilidades

Clase	Responsabilidad
Modulo	Almacenar los datos comunes de los módulos y definir la operación procesarCiclo().
ModuloEnergia	Generar energía y registrar sus ciclos y su producción histórica.
ModuloVuelo	Consumir energía, recolectar datos científicos y actualizar sus acumulados.
ModuloTierra	Consumir energía, descargar información y registrar el total descargado.
EstadoMision	Administrar la energía, los datos en órbita y el total histórico descargado.
Mision	Cargar, almacenar, listar, buscar y ordenar los módulos, además de avanzar ciclos y generar reportes.
VistaConsola	Mostrar el menú, recibir datos del usuario y presentar los resultados.

ControladorMision	Interpretar las opciones del menú y coordinar la vista con el modelo.
App	Crear el modelo, la vista y el controlador e iniciar el programa.

4. Descripción de las clases, atributos y métodos

   Clase abstracta: Modulo

   Atributo:

Atributo	Tipo	Visibilidad	Propósito
id	String	Privada	Identificar individualmente al módulo.
nombre	String	Privada	Guardar el nombre del módulo.
salud	int	Privada	Representar el porcentaje de salud.
activo	boolean	Privada	Indicar si el módulo puede operar.
costoConstruccion	double	Privada	Guardar el costo de construcción.

   Métodos:

Método	Visibilidad	Parámetros	Retorno	Propósito
Modulo()	Pública	id: String, nombre: String, salud: int, activo: boolean, costoConstruccion: double	No aplica	Inicializar los atributos comunes de los módulos.

getId()	Pública	Ninguno	String	Devolver el ID del módulo.
getNombre()	Pública	Ninguno	String	Devolver el nombre del módulo.
getSalud()	Pública	Ninguno	int	Devolver el porcentaje de salud.
isActivo()	Pública	Ninguno	boolean	Indicar si el módulo está activo.
getCostoConstruccion()	Pública	Ninguno	double	Devolver el costo de construcción.
setId()	Pública	id: String	void	Modificar el ID.
setNombre()	Pública	nombre: String	void	Modificar el nombre.
setSalud()	Pública	salud: int	void	Modificar la salud.
setActivo()	Pública	activo: boolean	void	Cambiar el estado activo.
setCostoConstruccion()	Pública	costoConstruccion: double	void	Modificar el costo de construcción.
procesarCiclo()	Pública y abstracta	estado: EstadoMision	String	Definir la operación polimórfica que cada

				clase hija debe implementar.
toString()	Pública	Ninguno	String	Devolver los datos comunes del módulo como texto.

Clase: EstadoMision

Atributos

Atributo	Tipo	Visibilidad	Propósito
energiaDisponible	double	Privada	Guardar la cantidad de energía disponible para los módulos.
datosEnOrbita	double	Privada	Guardar los datos recolectados que todavía no han sido descargados.
totalDatosDescargados	double	Privada	Registrar el total histórico de datos descargados por la misión.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
EstadoMision()	Pública	energiaInicial: double	No aplica	Crear un estado nuevo con energía inicial y los datos en cero.

EstadoMision()	Pública	energiaInicial: double, datosInicialesEnOrbita: double, totalInicialDescargado: double	No aplica	Crear el estado de una misión que ya está en progreso.
getEnergiaDisponible()	Pública	Ninguno	double	Devolver la energía disponible.
getDatosEnOrbita()	Pública	Ninguno	double	Devolver la cantidad de datos pendientes en órbita.
getTotalDatosDescargados()	Pública	Ninguno	double	Devolver el total histórico descargado por la misión.
agregarEnergia()	Pública	cantidad: double	void	Aumentar la energía disponible.
consumirEnergia()	Pública	cantidad: double	boolean	Descontar energía si existe una cantidad suficiente e indicar si fue posible.
agregarDatosEnOrbita()	Pública	cantidad: double	void	Agregar los datos recolectados por un

				módulo de vuelo.
descargarDatos()	Pública	cantidad: double	void	Restar datos de la órbita y agregarlos al total histórico descargado.
toString()	Pública	Ninguno	String	Devolver el estado actual de los recursos como texto.

Clase: ModuloEnergia

Atributos

Atributo	Tipo	Visibilidad	Propósito
energiaPorCiclo	double	Privada y constante (final)	Guardar la cantidad fija de energía generada en cada operación.
ciclosAcumulados	int	Privada	Registrar cuántos ciclos ha operado correctamente el módulo.
totalEnergiaGenerada	double	Privada	Registrar la cantidad histórica de energía generada.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito

ModuloEnergia()	Pública	id: String, nombre: String, salud: int, activo: boolean, costoConstruccion: double, energiaPorCiclo: double	No aplica	Crear un módulo nuevo con sus acumulados en cero.
ModuloEnergia()	Pública	Parámetros anteriores, ciclosAcumulados: int, totalEnergiaGenerada: double	No aplica	Crear un módulo que ya posee actividad histórica.
getEnergiaPorCiclo()	Pública	Ninguno	double	Devolver la energía generada por ciclo.
getCiclosAcumulados()	Pública	Ninguno	int	Devolver la cantidad de ciclos operados.
getTotalEnergiaGenerada()	Pública	Ninguno	double	Devolver el total histórico de energía generada.
procesarCiclo()	Pública	estado: EstadoMision	String	Generar energía si el módulo está activo y actualizar sus acumulados.

toString()	Pública	Ninguno	String	Devolver los datos generales y específicos del módulo de energía.

Clase: ModuloVuelo

Atributos

Atributo	Tipo	Visibilidad	Propósito
tipoInstrumento	String	Privada y constante (final)	Indicar si el instrumento es una cámara o un sensor.
datosPorCiclo	double	Privada y constante (final)	Guardar la cantidad fija de datos que recolecta por ciclo.
consumoEnergia	double	Privada y constante (final)	Guardar la energía necesaria para operar durante un ciclo.
ciclosAcumulados	int	Privada	Registrar la cantidad de ciclos en los que el módulo operó correctamente.
totalDatosRecolectados	double	Privada	Registrar el total histórico de datos recolectados.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
ModuloVuelo()	Pública	id: String, nombre: String, salud: int, activo: boolean, costoConstruccion: double, tipoInstrumento: String, datosPorCiclo: double, consumoEnergia: double	No aplica	Crear un módulo nuevo con sus acumulados en cero.
ModuloVuelo()	Pública	Parámetros anteriores, ciclosAcumulados: int, totalDatosRecolectados: double	No aplica	Crear un módulo que ya posee actividad histórica.
getTipoInstrumento()	Pública	Ninguno	String	Devolver el tipo de instrumento.
getDatosPorCiclo()	Pública	Ninguno	double	Devolver la cantidad de datos recolectados por ciclo.
getConsumoEnergia()	Pública	Ninguno	double	Devolver la energía necesaria para operar.
getCiclosAcumulados()	Pública	Ninguno	int	Devolver la cantidad de ciclos

				operados correctamente.
getTotalDatosRecolectados()	Pública	Ninguno	double	Devolver el total histórico de datos recolectados.
procesarCiclo()	Pública	estado: EstadoMision	String	Comprobar las condiciones, consumir energía y recolectar datos.
toString()	Pública	Ninguno	String	Devolver los datos generales y específicos del módulo de vuelo.

Clase: ModuloTierra

Atributos

Atributo	Tipo	Visibilidad	Propósito
estacion	String	Privada y constante (final)	Guardar el nombre de la estación que recibe la información.
capacidadDescarga	double	Privada y constante (final)	Indicar la cantidad máxima de datos que puede descargar por ciclo.

energiaPorDescarga	double	Privada y constante (final)	Guardar la energía necesaria para realizar una descarga.
totalDescargado	double	Privada	Registrar el total histórico descargado por el módulo.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
ModuloTierra()	Pública	id: String, nombre: String, salud: int, activo: boolean, costoConstruccion: double, estacion: String, capacidadDescarga: double, energiaPorDescarga: double	No aplica	Crear un módulo nuevo con su total descargado en cero.
ModuloTierra()	Pública	Parámetros anteriores, totalDescargado: double	No aplica	Crear un módulo que ya posee datos descargados históricamente.
getEstacion()	Pública	Ninguno	String	Devolver el nombre de la estación.
getCapacidadDescarga()	Pública	Ninguno	double	Devolver la capacidad máxima de descarga por ciclo.

getEnergiaPorDescarga()	Pública	Ninguno	double	Devolver la energía requerida para una descarga.
getTotalDescargado()	Pública	Ninguno	double	Devolver el total histórico descargado por el módulo.
procesarCiclo()	Pública	estado: EstadoMision	String	Comprobar las condiciones, consumir energía y descargar los datos disponibles.
toString()	Pública	Ninguno	String	Devolver los datos generales y específicos del módulo de tierra.

Clase: Mision

Atributos

Atributo	Tipo	Visibilidad	Propósito
modulos	List<Modulo>	Privada y constante (final)	Almacenar todos los módulos en una única lista polimórfica.
estado	EstadoMision	Privada y constante (final)	Mantener los recursos compartidos y acumulados de la misión.
numeroDeCiclos	int	Privada	Registrar cuántos ciclos ha avanzado el simulador.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
Mision()	Pública	Ninguno	No aplica	Crear la lista y el estado inicial, establecer el número de ciclos y cargar los módulos.
cargarModulosIniciales()	Privada	Ninguno	void	Crear los diez módulos iniciales y agregarlos a la lista polimórfica.
getModulos()	Pública	Ninguno	List<Modulo>	Devolver la lista de módulos.

getEstado()	Pública	Ninguno	EstadoMision	Devolver el estado actual de la misión.
listarModulos()	Pública	Ninguno	String	Recorrer y mostrar todos los módulos con sus características.
buscarModulo()	Pública	criterio: String	Modulo	Buscar un módulo por ID o nombre y devolverlo; retorna null si no existe.
catalogoOrdenadoPorCosto()	Pública	Ninguno	String	Crear y mostrar una copia de la lista ordenada por costo.
avanzarCiclo()	Pública	Ninguno	String	Dar a todos los módulos la oportunidad de operar y mostrar los recursos resultantes.
generarResumenComunicaciones()	Pública	Ninguno	String	Calcular y mostrar la información correspondiente a los

				módulos de tierra.

Clase: VistaConsola

Atributos

Atributo	Tipo	Visibilidad	Propósito
scanner	Scanner	Privada y constante (final)	Leer la información ingresada por el usuario desde la consola.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
VistaConsola()	Pública	Ninguno	No aplica	Crear el objeto Scanner utilizado para leer datos.
mostrarMenu()	Pública	Ninguno	void	Mostrar las opciones disponibles en el simulador.
LeerOpcion()	Pública	Ninguno	int	Leer la opción del menú y validar que sea un número entero.
leerTexto()	Pública	mensaje: String	String	Mostrar una indicación y devolver el texto ingresado sin espacios externos.
mostrarMensaje()	Pública	mensaje: String	void	Mostrar en la consola un mensaje o resultado.

Clase: ControladorMision

Atributos

Atributo	Tipo	Visibilidad	Propósito
mision	Mision	Privada y constante (final)	Mantener la referencia al modelo para solicitar las operaciones del sistema.
vista	VistaConsola	Privada y constante (final)	Mantener la referencia a la vista para recibir datos y mostrar resultados.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
ControladorMision()	Pública	mision: Mision, vista: VistaConsola	No aplica	Recibir y guardar las referencias del modelo y de la vista.
iniciar()	Pública	Ninguno	void	Mostrar repetidamente el menú y ejecutar la opción seleccionada hasta que el usuario decida salir.
buscarModulo()	Privada	Ninguno	void	Solicitar el criterio de búsqueda, consultar la misión y mostrar el resultado.

Clase: App

Atributos

Atributo	Tipo	Visibilidad	Propósito
No posee atributos	No aplica	No aplica	La clase solamente contiene el punto de inicio del programa.

Métodos

Método	Visibilidad	Parámetros	Retorno	Propósito
main()	Pública y estática	args: String[]	void	Crear la misión, la vista y el controlador, y luego iniciar el programa.

5. Inicialización de los objetos y colaboración entre constructores

Clase	Forma de inicialización
Modulo	Su constructor recibe e inicializa los datos comunes: ID, nombre, salud, estado activo y costo de construcción.
ModuloEnergia	Recibe los datos comunes y la energía por ciclo. Utiliza super() para inicializar la parte heredada de "Modulo". Puede iniciar sus acumulados en cero o recibir valores históricos.
ModuloVuelo	Recibe los datos comunes, el instrumento, los datos por ciclo y el consumo energético. Utiliza super() para inicializar los atributos heredados y puede comenzar con acumulados históricos.
ModuloTierra	Recibe los datos comunes, la estación, la capacidad y la energía por descarga. Utiliza super() y permite iniciar el total descargado en cero o con un valor histórico.
EstadoMision	Puede iniciar solamente con energía y dejar los datos en cero, o recibir energía, datos en órbita y total descargado para representar una misión en progreso.
Mision	Crea la lista, el estado inicial, establece el contador de ciclos en cero y llama a cargarModulosIniciales().
VistaConsola	Crea el objeto Scanner para leer la entrada del usuario.

ControladorMision	Recibe desde “App” los objetos “Mision” y “VistaConsola” y guarda sus referencias.
App	Crea los objetos principales, los conecta mediante el constructor de “ControladorMision” e inicia el sistema.

6. Herencia y polimorfismo

    - “Modulo” es la clase padre abstracta de “ModuloEnergia”, “ModuloVuelo” y “ModuloTierra”. Las clases hijas heredan los atributos y métodos comunes, evitando repetir código.
    - El polimorfismo se aplica mediante List<Modulo>, que permite almacenar los tres tipos de módulos en una sola lista. Cada clase hija sobrescribe procesarCiclo() con su propia operación: generar energía, recolectar datos o descargarlos.
    - Cuando “Mision” ejecuta modulo.procesarCiclo(estado), Java selecciona automáticamente el comportamiento correspondiente al tipo real del objeto. También se sobrescribe toString() para mostrar la información específica de cada módulo.

7. Aplicación del patrón MVC

    - El modelo está formado por “Modulo”, sus tres clases hijas, “EstadoMision” y “Mision”. Estas clases almacenan los datos y contienen la lógica del simulador.
    - La vista corresponde a “VistaConsola”. Su responsabilidad es mostrar el menú, leer la información ingresada y presentar los resultados.
    - El controlador es “ControladorMision”. Recibe las opciones de la vista, solicita al modelo la operación correspondiente y envía el resultado nuevamente a la vista.
    - “App” crea estos componentes y los conecta. Esta separación evita colocar toda la lógica en main y permite que cada clase tenga una responsabilidad definida.

8. Relación entre las clases

Clases relacionadas	Tipo de relación	Explicación

“Modulo” y “ModuloEnergia”	Herencia	“ModuloEnergia” hereda los datos y métodos comunes de “Modulo”.
“Modulo” y “ModuloVuelo”	Herencia	“ModuloVuelo” hereda los datos y métodos comunes de “Modulo”.
“Modulo” y “ModuloTierra”	Herencia	“ModuloTierra” hereda los datos y métodos comunes de “Modulo”.
“Mision” y “Modulo”	Composición	“Mision” crea y administra la lista de módulos que forman parte de ella.
“Mision” y “EstadoMision”	Composición	“Mision” crea y conserva el objeto que representa sus recursos.
“Modulo” y “EstadoMision”	Dependencia	Los módulos reciben el estado como parámetro para consultar y modificar los recursos durante un ciclo.
“ControladorMision” y “Mision”	Asociación	El controlador mantiene una referencia al modelo y solicita sus operaciones.
“ControladorMision” y “VistaConsola”	Asociación	El controlador mantiene una referencia a la vista para recibir y mostrar información.
“VistaConsola” y Scanner	Composición	La vista crea y utiliza el objeto que permite leer datos desde la consola.
“App” con las clases principales	Dependencia	“App” crea “Mision”, “VistaConsola” y “ControladorMision” para iniciar el sistema.

Diagrama UML

